CSC490A2 - Group 8

Isaac Abell - 1008831329, Thomas(Yehyun) Lee - 1008992217,
Zeling (Zoey) Zhang - 1007714779, Jessica Zhang- 1008494130

Table of Contents

Part 1: Architecture Review..... 1
1.1 Architecture Diagram..... 1
1.2 Architecture Explanation..... 2
1.3 Proposed Architecture Upgrades..... 2
Table1. Summary of Proposed Changes..... 2
Detailed Analysis of Changes..... 5
Part 2: Ablations on a “Picochat”..... 7
2.1 Picochat Baseline Configuration..... 7
2.2 Ablation choices & hypotheses..... 7
2.2 Ablation Results..... 8
2.3 Commentary..... 9
2.3.1 Impact of Changes..... 9
2.3.2 W&B tracking: what we compared..... 10
2.3.3 Cost discipline and what we learned from cost..... 11
2.3.4 Scaling Predictions..... 11
Part 3: Extending the Context Window..... 12
3.1 Shorter Limit Justification..... 12
3.2 Custom Evaluation..... 12
3.3 Training Dynamics for 512 vs 2048 Seq. & W&B Insights..... 13
3.3.1 Token Throughput vs. Sequence Length..... 14
3.3.2 General Capability Improvements..... 14
3.3.3 Training Stability..... 14
Part 4: Training Your Final Nanochat..... 15
4.1 Final Configuration..... 15
4.2 Scaling Law Prediction..... 15
4.3 Final Model Summary..... 18
4.4 Emergent Abilities (Nanochat vs. Picochat)..... 19
Works Cited..... 26

Part 1: Architecture Review

1.1 Architecture Diagram

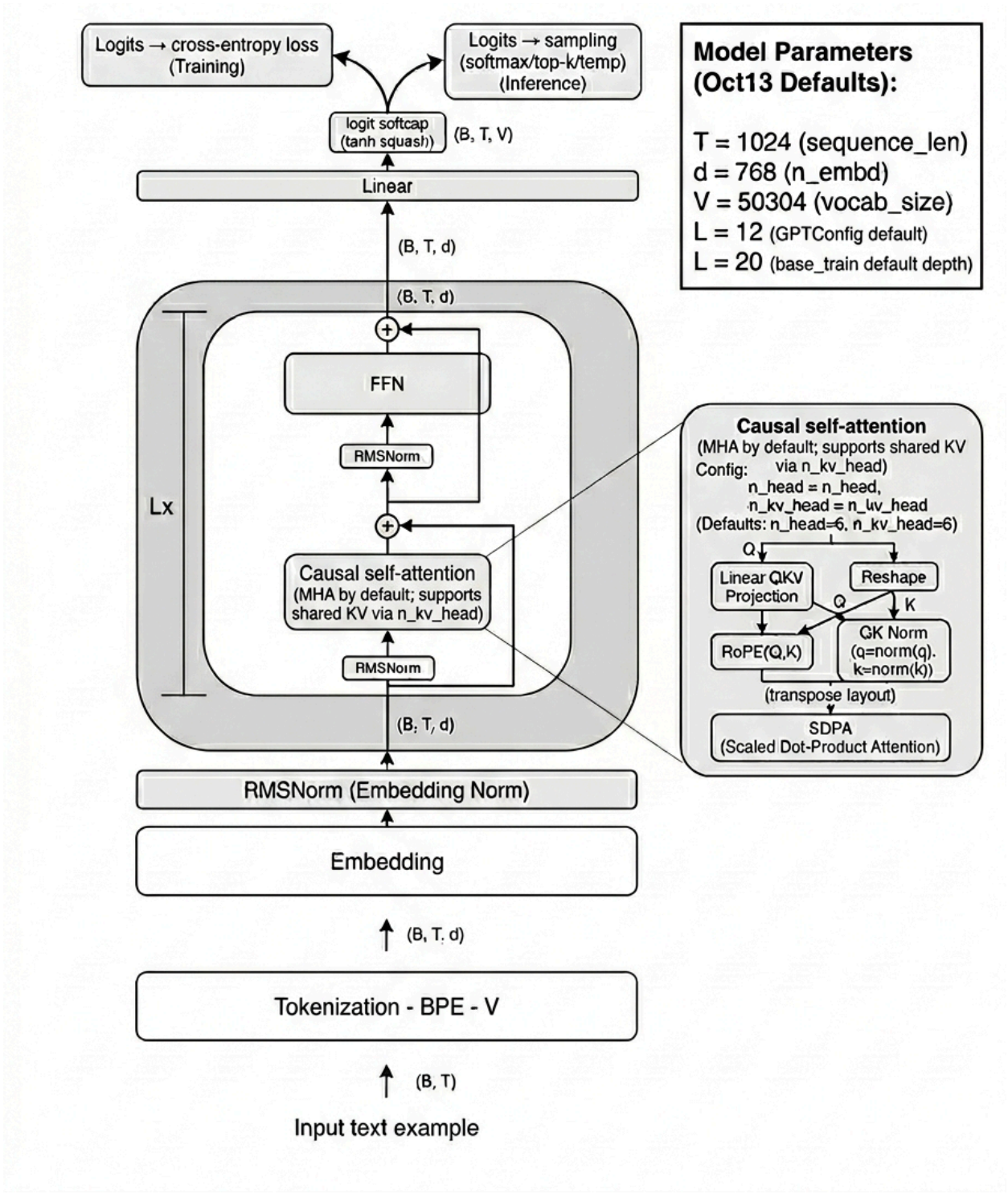


Figure 1. Nanochat Architecture based on Oct. Release.

1.2 Architecture Explanation

Oct 13 nanochat is a lean decoder-only GPT stack: BPE-tokenized IDs → token embedding → RMSNorm (post-embed) → L pre-norm Transformer blocks → final RMSNorm → untied linear LM head → logits, with a tanh "logit softcap" applied before loss/sampling.

Each block uses pre-norm residual wiring ($x + Attn(RMSNorm(x)), x + MLP(RMSNorm(x))$). Attention is causal SDPA with RoPE on (Q, K) and QK norm for stabilization; it defaults to standard MHA (e.g., $n_head = 6, n_kv_head = 6$) but supports shared KV via n_kv_head for KV-reduction variants. The MLP is a simple 4× expansion + ReLU². Across the model, design choices emphasize minimalism and stability: functional RMSNorm with no learnable γ/β, bias-free linear layers, and explicit separation of embedding/unembedding weights. Depth is configuration-driven (e.g., gpt config default L=12 vs base_train default depth=20).

1.3 Proposed Architecture Upgrades

To modernize the newest Nanochat architecture (Feb 2026), we propose three architectural changes backed by recent literature. These modifications aim to balance the trade-offs between computational efficiency, training speed, and overall model performance.

Table1. Summary of Proposed Changes

Proposed Change	Motivation	Technical Details	Potential Impact on Performance
1. SwiGLU Activation	Nanochat's baseline FFN uses a single-stream ReLU MLP, a deliberate cost-saving choice that keeps training fast under a tight budget, but it also limits token-conditional feature routing in the part of the Transformer that provides much of the per-layer transformation power, especially in shallow configurations. Since ReLU is a fixed nonlinearity with no adaptive gating, the model lacks an internal mechanism to selectively suppress or amplify intermediate features based on the current token/context; instead, it must	Replace the existing ReLU activation in the FFN with a gated MLP where a gate projection modulates a value projection (with Swish Gated Linear Unit). (Shazeer, 2020; Touvron et al., 2023)	**Positive:** Tends to improve perplexity and downstream robustness at fixed training tokens by adding token-conditional feature selection inside the FFN; often improves optimization stability and "quality-per-token," which is especially valuable in shallow configurations. (Shazeer, 2020) **Negative:** Adds an extra projection in the FFN: at fixed hidden expansion, this increases MLP parameters/compute

	“buy” the same effect indirectly by spending more tokens or scaling parameters to reach a similar loss. Replacing the FFN with a gated variant (SwiGLU) will address this gap by adding an explicit gate that multiplicatively modulates a value stream, turning the FFN into a form of conditional computation (the model learns when to pass features through and when to block them). This tends to improve optimization efficiency (more stable gradients) and increases effective expressivity per layer without requiring additional depth, which is why it often translates into lower perplexity and stronger downstream behavior at a fixed token budget, assuming we accept the extra MLP compute and implement it efficiently. (Shazeer, 2020; Touvron et al., 2023)		and can reduce throughput if the run is MLP-bound; the practical gain depends on whether you do a **capacity-increase** (keep expansion) or a **compute-matched swap** (shrink expansion), and whether the implementation remains on efficient GEMM paths rather than introducing unfused overhead. (Shazeer, 2020)
2. ALiBi (Attention with Linear Biases)	Nanochat’s current positional handling is optimized for performance within the training context window, but lightweight decoder-only models often fail sharply when evaluated on contexts longer than they were trained on: attention becomes less reliable at retrieving earlier information, coherence degrades with distance, and these errors compound	Replace Rotary Positional Embeddings (RoPE) by removing positional embeddings and adding a static, non-learned linear bias to attention	**Positive:** Enables “train short, test long” behavior more reliably by imposing a simple distance prior in the scoring function itself; supports **zero-shot length extrapolation** and often degrades more gracefully as context grows beyond

	because decoding is autoregressive. This exposes a gap in nanochat's current setup: standard positional schemes can implicitly "lock" attention behavior to the trained length regime, and any existing structural heuristics (e.g., restricting attention span) constrain *which* tokens are even eligible, but do not guarantee stable, length-agnostic attention scoring when the model is pushed beyond its training horizon. ALiBi directly targets this extrapolation failure mode by injecting a head-wise linear distance bias into the attention logits, imposing a simple inductive prior that remains well-defined for arbitrary sequence lengths: nearby tokens are penalized less and distant tokens more, using fixed head-specific slopes. Because this positional signal is implemented as a rule inside the attention scores rather than as learned embeddings or fixed rotation frequencies tied to a maximum length, the same mechanism continues to apply when context increases, enabling "train short, test long" behavior and supporting zero-shot length extrapolation, being valuable for a small model where we want longer-context robustness	scores based on token distance. (Press et al., 2021).	the training window. (Press et al., 2021) **Negative:** Does not change dense attention's O(T2)O(T^2)O(T2) compute; any speed benefit is implementation-dependent and can be reversed if the bias is applied in a way that breaks optimized attention kernels (e.g., forcing a slower path via explicit bias materialization). Also, when combined with windowing, the linear distance prior can further tilt attention toward the newest tokens, which may reduce recall of information near the far edge of the available span unless evaluated and tuned carefully. (Press et al., 2021)

	without additional finetuning. (Press et al., 2021)		
3. Selective Attention	**Bottlenecks in long-context decoding-only Transformers:** Training: attention remains **$O(T^2)$** in sequence length and Inference: KV-cache grows O(T) with context length and often dominates memory at long horizons. **Why nanochat's current mitigations are incomplete:** Feb nanochat already includes GQA and layer-wise sliding-window patterns to reduce effective attention span (Karpathy, 2025). These are structural heuristics. They reduce eligible context *by position*, not by *utility*. Thus, they can discard rare yet important long-range tokens, which may hurt in-context behaviour.	Keep the standard pipeline and add a selectivity signal that downweights / masks attention contributions from a subset of past tokens, i.e., a structured modification to attention logits or weights so only a budgeted subset remains effectively attendable.	**Positive:** enables reducing the effective context buffer size with meaningful memory/compute savings at matched quality. **Negative:** the practical win depends on an implementation that retains FA3/SDPA fast paths; naïve sparsification can erase the speedup; benefits are most visible when context is long and memory-constrained; short-context pico runs may show smaller/noisier gains.

Detailed Analysis of Changes

1. SwiGLU instead of ReLU

- **Primary Paper:** GLU Variants Improve Transformer (Shazeer, 2020) - arXiv:2002.05202
- **Supporting Context:** LLaMA: Open and Efficient Foundation Language Models (Touvron et al., 2023) - arXiv:2302.13971
- **Justification:** Several modern LLMs (such as the LLaMA family) have widely adopted SwiGLU over standard ReLU. The original Nanochat opted for ReLU due to their primary motivation being training a model that rivaled GPT-2 for under $100. While ReLU

requires fewer computations (making it faster and reducing the total training time), swapping it for SwiGLU should lead to better perplexity and performance on downstream tasks, making it a good upgrade if we are willing to accept slightly longer training times (assuming it actually leads to improved performance).

2. ALiBi instead of RoPE

- **Primary Paper:** Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation (Press et al., 2021) - arXiv:2108.12409
- **Justification:** While the current model utilizes RoPE, ALiBi presents a potential alternative for handling context length. Instead of adding positional information at the embedding layer, ALiBi penalizes attention scores between distant tokens with a linear bias. This allows the model to achieve "zero-shot length extrapolation," processing sequences longer than it was trained on, highly benefiting a lightweight model like Nanochat.

3. Selective Attention

- **Primary Paper:** Selective Attention Improves Transformer (Leviathan, Kalman, & Matias, 2025)
- **Justification:** Selective Attention could be a nanochat upgrade as it directly targets the same bottleneck nanochat is already optimizing, attention's context management, but replaces fixed heuristics (e.g., sliding windows) with a principled mechanism that learns to suppress unneeded context (Karpathy, 2025). Leviathan, Kalman, and Matias (2025) frame it as a *parameter-free* change that adds negligible compute while consistently improving language modeling and downstream performance across model sizes and context lengths, and further enables shrinking the attention context buffer for meaningful memory/compute savings. In a nanochat setting where training budget and implementation complexity are tightly constrained, this provides an unusually favorable trade-off: it is orthogonal to capacity-oriented changes (like FFN upgrades) and positional schemes (like ALiBi), yet can improve both quality and efficiency, which is precisely the kind of upgrade that benefits from systematic ablation in a small-model regime before scaling.

Part 2: Ablations on a “Picochat”

2.1 Picochat Baseline Configuration

Modifications. We downscaled the default nanochat configuration by reducing **depth from 24 → 12** and **num_shards from 240 → 40**, yielding a “picochat” setting suitable for rapid ablation cycles.

Justification. Halving depth reduces the model from **~768M** to **~125M** parameters (roughly GPT-2 small scale), enabling proportionally faster iterations while remaining large enough to produce stable metric deltas. The **40-shard** data budget (≈2B tokens) corresponds to **~16× tokens per parameter**, which is in the vicinity of Chinchilla-style scaling heuristics and helps ensure the model sees sufficient data to learn coherent behavior rather than trivially memorizing or underfitting under an overly small token budget.

2.2 Ablation choices & hypotheses

We selected **SwiGLU** and **ALiBi** because they represent two different categories of architectural bets: a **localized capacity/optimization upgrade** in the feed-forward block (SwiGLU), and a **positional inductive-bias change** aimed specifically at long-context behavior (ALiBi). SwiGLU primarily targets quality-per-token by improving the FFN’s internal routing and training dynamics via gating; we expected gains to show up in standard pretraining metrics (loss/perplexity, downstream eval proxies) with predictable overhead because it does not alter attention structure or the attention backend’s execution path.

For ALiBi, we deliberately separated two hypotheses that can easily get conflated: **(A) “short-context quality/cost”** vs **(B) “zero-shot length extrapolation.”** Our initial Part 2-style comparisons (standard pretrain/eval metrics) primarily test (A), and in our setting, ALiBi did not show a consistent advantage. However, ALiBi’s central claim in the literature is (B): models trained on shorter sequences can generalize more gracefully to longer contexts without additional fine-tuning. To evaluate that claim directly, we added an additional targeted ablation beyond the standard pretrain metrics: we trained both RoPE and ALiBi variants only at seq_len=512 for the same number of steps (1000), then evaluated *both checkpoints* on a **Needle-in-a-Haystack retrieval test** at **512 and 1030 tokens without any continued training** (and across multiple placements/seeds). This isolates whether ALiBi reduces degradation when the test length exceeds the training length, independent of any improvements that would come from further training at longer contexts.

2.2 Ablation Results

Figure 2. W&B training diagnostics for picochat (d12) ablations comparing baseline, SwiGLU, and ALiBi: token throughput (tok/s) and MFU show ALiBi's sustained ~2× efficiency drop, while loss and LR curves remain comparable, indicating the ALiBi cost increase is driven primarily by a slower attention backend/kernel path rather than unstable optimization.

Table 2. Picochat ablation results (bpb/CORE/MFU/time/cost)

Model	Architecture Changes	Val bpb	CORE metric	MFU	Training Time (min)	Compute Cost ($)
Model 1 (Baseline)	Standard Picochat (Depth 12, 40 Shards)	0.9057	0.124	42.1%	9.27	3.10

Model 2	Picochat + SwiGLU	0.9064	0.137	40.1%	9.62	4.67
Model 3	Picochat + ALiBi	0.9119	0.136	21.7%	17.90	6.77

Table 3. Needle-in-a-Haystack ablation results (exact-match retrieval accuracy by context length and needle placement)

Model (trained @512, 1000 steps)	Eval length	Overall exact-match accuracy	Placement 0.00	Placement 0.25	Placement 0.50	Placement 0.75	Placement 1.00
RoPE	512	32% (8/25)	0% (0/5)	20% (1/5)	20% (1/5)	20% (1/5)	100% (5/5)
RoPE	1030	16% (4/25)	0% (0/5)	0% (0/5)	0% (0/5)	40% (2/5)	40% (2/5)
ALiBi	512	8% (2/25)	0% (0/5)	0% (0/5)	0% (0/5)	0% (0/5)	40% (2/5)
ALiBi	1030	20% (5/25)	0% (0/5)	0% (0/5)	0% (0/5)	0% (0/5)	100% (5/5)

2.3 Commentary

2.3.1 Impact of Changes

As summarized in Table 2 and diagnosed via Fig.2, the results largely **confirm** the SwiGLU hypothesis and **weaken** the ALiBi one. SwiGLU provides a clear CORE improvement with minimal runtime impact, indicating a favorable quality/efficiency trade-off. By contrast, ALiBi does not produce a quality advantage over SwiGLU and induces a substantial throughput/MFU regression, suggesting that in our stack, it likely disrupts the optimized attention kernel path. We hypothesize this is due to AliBi not being compatible with Flash Attention 3, leading to longer calculations due to the model defaulting on the standard attention calculations instead of the optimized Flash Attention calculations. Overall, this supports the broader conclusion that attention/positional changes are not purely modeling decisions: their practicality at scale is gated by whether they preserve fast-path attention implementations.

1. **SwiGLU (clear quality win at small runtime overhead)**
   - Relative to the picochat baseline shown in Table 2, CORE improves from **0.124** to **0.137** (**+10.5%**), while val bpb is essentially unchanged (0.9057 to 0.9064).

- Runtime stays nearly flat (**9.27 to 9.62 min**). MFU drops slightly (42.1% to 40.1%), consistent with a modestly heavier MLP compute, but not enough to matter operationally at this scale (Table 2).
- **Interpretation:** SwiGLU tends to increase expressivity/gradient flow in the MLP block; at this budget, the gain is more pronounced in our **reasoning-ish** metric (CORE) than in raw bpb. This is exactly the kind of "cheap improvement" that survives into larger runs because it doesn't require changing the attention kernel path.

2. **ALiBi (efficiency regression; quality not better than SwiGLU)**
   - Val bpb gets **worse** (0.9057 → **0.9119**) compared to the baseline model (Table 2). CORE rises vs baseline (0.124 → 0.136), but is **not better than SwiGLU** (0.136 vs 0.137).
   - Training time nearly **doubles** (**9.27** → **17.90 min**), and MFU collapses (**42.1%** → **21.7%**).
   - **Interpretation:** The evidence suggests that ALiBi causes the model to execute attention in a less-optimized regime (e.g., by losing a fused fast-path kernel), leading to lower hardware utilization and a large runtime penalty. On modern stacks, this pattern commonly occurs when introducing attention biases / positional mechanisms that change kernel dispatch or require extra operations/layouts that are not supported by the most optimized fused attention implementations (cf. ALiBi–FlashAttention discussions in prior work). The net result here is unfavorable: ALiBi substantially increases costs without delivering a quality advantage over SwiGLU.
   - We ran a controlled ablation comparing RoPE and ALiBi positional encodings on a d12 Transformer trained for 1,000 steps at sequence length 512, holding all other hyperparameters constant (model width, data shards, optimizer, and learning-rate schedule). We then evaluated both checkpoints *without any additional training* on a Needle-in-a-Haystack retrieval task at **512 tokens** (in-distribution) and **1030 tokens** (out-of-distribution), with the needle placed at **five relative positions** (0.0, 0.25, 0.5, 0.75, 1.0) and **5 random seeds per condition**. As shown in table 3, RoPE achieved higher in-distribution retrieval accuracy (**32%**) and showed *limited but non-zero* non-recency retrieval at placements **0.25–0.75** (each 1/5), while ALiBi achieved only **8%** (2/25) at 512 and succeeded **exclusively at the most recent placement** (1.0: **40%**, 2/5), suggesting a strong recency heuristic rather than distance-robust retrieval. When evaluated out-of-distribution at 1030 tokens, RoPE degraded in aggregate (**32%** → **16%**) and lost all early/mid placements, with remaining successes concentrated at **0.75 and 1.0** (each **40%**, 2/5). ALiBi's aggregate accuracy **increased** (**8%** → **20%**), but this improvement was entirely driven by becoming **perfect at placement 1.0** (**100%**, 5/5) while remaining **0%** at all earlier placements. Thus, while ALiBi appears to "degrade less" in aggregate, our placement breakdown does **not** support the intended interpretation of improved long-context generalization: under this training budget, ALiBi never learns non-recency retrieval, and its apparent OOD gain reflects a stronger reliance on recency rather than the "train short, test long" retrieval behavior emphasized by ALiBi's original framing.

2.3.2 W&B tracking: what we compared

In Weights & Biases, we tracked and overlaid **training curves** (loss and bpb vs. steps) to compare convergence behaviour, **eval metrics** (validation bpb and CORE vs. steps) to measure quality deltas, and **system metrics** (step time, tokens/sec, and MFU) to attribute differences to hardware utilization versus optimization effects. This makes the ALiBi outcome particularly easy to diagnose. Across training, ALiBi exhibits a sustained reduction in token throughput and MFU (Fig. 2a–b), implying systematically higher per-step compute time, while its loss curve follows a similar trajectory to the baseline and SwiGLU runs (Fig. 2d). Taken together with the aggregate metrics in Table 2, the evidence points to a systems-level efficiency regression, likely due to attention backend/kernel-path dispatch rather than insufficient optimization progress that would be resolved by additional training.

2.3.3 Cost discipline and what we learned from cost

Even at the picochat scale, the cost differences are material: **$3.10** baseline vs **$4.67** for SwiGLU and **$6.77** for ALiBi (Table 2). The key lesson is that architectural changes can shift **kernel efficiency** as much as modeling capacity: when a modification alters the attention execution path, reduced MFU and throughput (Fig. 1a–b) can dominate total training cost even when parameter count is similar.

2.3.4 Scaling Predictions

- **SwiGLU: likely scales well.** Gated MLP activations are widely used in modern LLM architectures because they often improve quality without materially changing the attention execution path, and therefore tend to preserve highly optimized attention kernels. Given that SwiGLU delivers a clear CORE gain at picochat scale with only a minor MFU/runtime impact, we expect the quality benefit to persist, potentially becoming more pronounced as model capacity increases, while the efficiency overhead remains relatively bounded.
- **ALiBi: strongly contingent on implementation details and kernel compatibility**. In our setup, ALiBi induces a large and sustained efficiency regression (MFU approximately halved), which would compound directly into substantial wall-clock and dollar cost at larger scale. Consequently, the appropriate next step is not to increase model size, but to first validate whether ALiBi is forcing a slower attention backend/kernel-path dispatch. If so, we would (i) instrument kernel selection and attention backend usage, (ii) test an implementation that preserves fused fast-path attention, and (iii) benchmark against positional alternatives with known compatibility with FlashAttention-style kernels (e.g., RoPE variants), selecting the approach that achieves quality gains without sacrificing throughput.

Part 3: Extending the Context Window

3.1 Shorter Limit Justification

We have chosen to pre-train our Picochat model on a smaller context window of 512 tokens to begin with. This is because pre-training LLMs from scratch on such large context windows is computationally expensive due to the attention mechanism's quadratic scaling with the length of the input sequences: O(n^2). This way, we can quickly iterate and teach the model the language's basic grammar, syntax, and semantics at a much lower computational cost.

As per the paper "Train Short, Test Long" (Press et al., 2021), which introduced the ALiBi model, it has been demonstrated that models with the right kind of positional encoding can be pre-trained on short sequences and then efficiently generalized to longer sequences. Moreover, the ALiBi model is particularly well-suited to this kind of pre-training due to the way it adds a static linear bias to the model, completely avoiding the need to pre-train the positional embeddings. We have pre-trained the model on 1000 steps at length 512, teaching it the basic capabilities, and then increased the length to 2048 tokens and pre-trained the model further for another 1000 steps to increase the model's effective receptive field.

3.2 Custom Evaluation

- **Eval Task**
  To test the model's long-context retrieval capabilities, we created a custom "Needle in a Haystack" evaluation. Our model was given a prompt that contained a specific secret phrase ("The secret password is: *ALIBI_IS_AWESOME.*") embedded in a large amount of "filler" text ("*The quick brown fox…*"). The total length of the prompt was 1030 tokens, which is much longer than the 512 token context window used in the first checkpoint, but still within the 2048 token context window used in the second checkpoint. The model is given the task of retrieving the secret password at the very end of the prompt.

```
2026-03-01 22:02:48,071 - nanochat.checkpoint_manager - INFO - Loading model from /vol/nanochat_cache/base_checkpoints/pico-part3 with step 2000
2026-03-01 22:02:48,747 - nanochat.checkpoint_manager - INFO - Building model with config: {'sequence_len': 2048, 'vocab_size': 32768, 'n_layer': 1
 6, 'n_embd': 768, 'window_pattern': 'SSSL'}
Autodetected device type: cpu

--- Evaluating Checkpoint at Step 1000 ---
Prompt length: 1030 tokens
Model Output: The secret password is the secret password. The secret password is the secret password. The secret password is
BPB on this context: 0.3687

--- Evaluating Checkpoint at Step 2000 ---
Prompt length: 1030 tokens
Model Output: The secret password is: ALIBI_IS_AWESOME. The secret password is
BPB on this context: 0.3420

======================================
FINAL COMPARISON
======================================
Step 1000 (Short Context (512)): The secret password is the secret password. The secret password is the secret password. The secret password is
Step 2000 (Long Context (2048)): The secret password is: ALIBI_IS_AWESOME. The secret password is
```

Figure 3. Needle in a Haystack eval output.

- **Results**
  - **Checkpoint 1 (512 seq) Performance:** Failed. The model was only able to output a repetitive hallucination of the following: "The secret password is the secret password. The secret password is the secret password. The secret password is". The BPB of the prompt was 0.3687.

- **Checkpoint 2 (2048 seq) Performance:** Success. The model was able to retrieve the target phrase and output the following: “The secret password is: ALIBI_IS_AWESOME. The secret password is”. The BPB of the prompt was reduced to 0.3420.

- **Conclusion**
  It is evident from the results that the proposed method of context extension was a resounding success. It was seen that the Checkpoint 1 model, which was only capable of processing sequences of length 512, was not able to effectively utilize the information that was placed more than 500 tokens deep into the prompt. On the other hand, the Checkpoint 2 model was able to adapt to the new prompt and not only retrieve the exact target phrase but also achieve a reduced BPB, indicating that it was more effective overall at modelling the long sequence length distribution.

3.3 Training Dynamics for 512 vs 2048 Seq. & W&B Insights

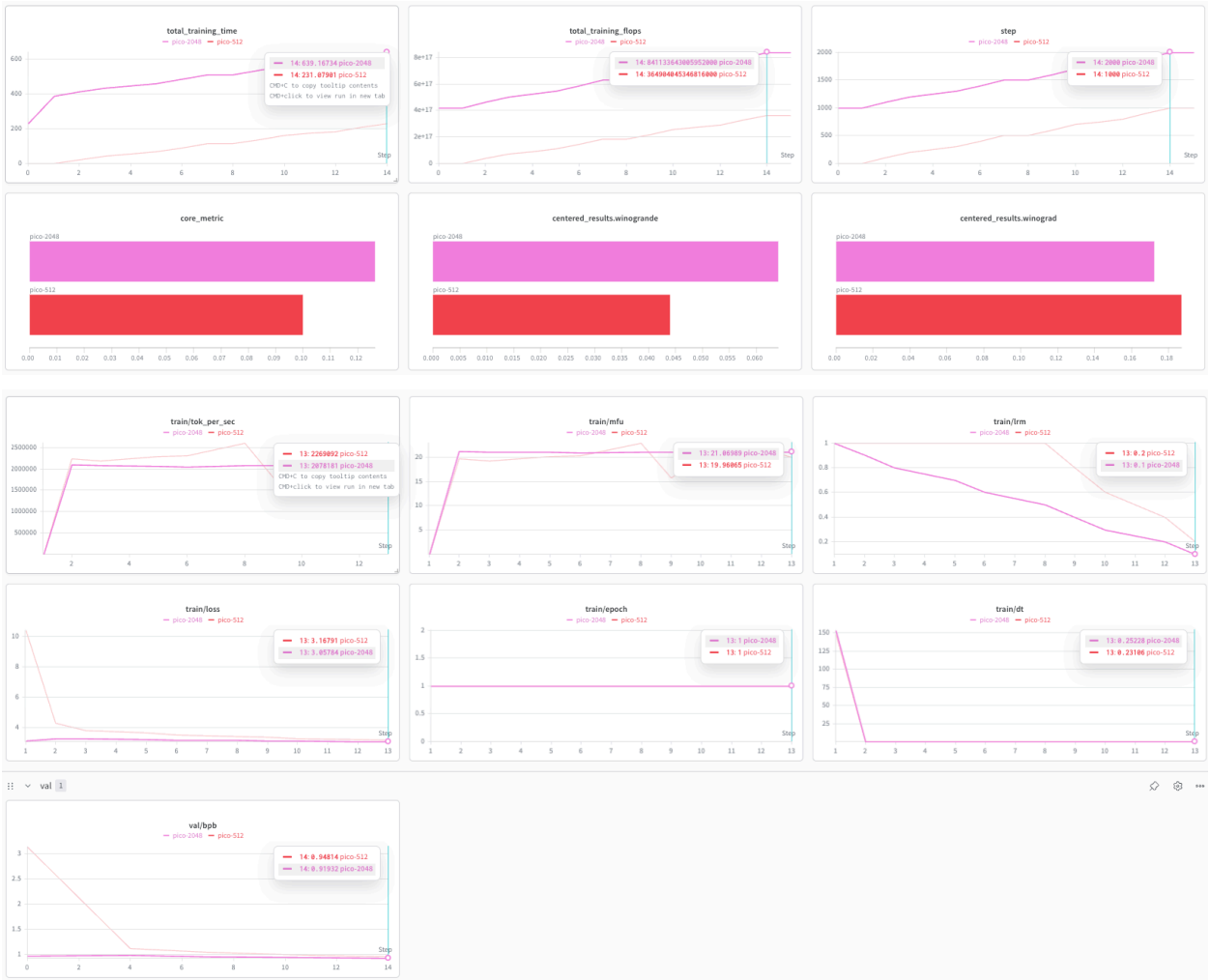


Figure 4. Weights & Biases charts comparing 512 vs 2048 sequence training.

Observing the two training phases for 512-context and 2048-context, using the Weights & Biases figure above gave us a couple of interesting insights.

3.3.1 Token Throughput vs. Sequence Length

One of the surprising observations from the *train/tok_per_sec* chart is the lack of a substantial decrease in the speed of the model when we increase the context length four times. We observe a speed of ~2.3M tokens/sec during the 512-context training phase and a speed of ~2.07M tokens/sec during the 2048-context training phase. Similarly, the utilization of the Model FLOPs Utilization (MFU) also seems constant at ~20-21%. We use highly optimized FlashAttention, so the O(n^2) cost of the attention mechanism is still quite fast for our sequence length. The major cost is still the O(n · d^2) dense linear layers (MLPs/QKV projections), where we process the same number of tokens overall per batch regardless of their sequence length. Training at length 512 is thus only accelerating the speed of sequences processed per batch.

3.3.2 General Capability Improvements

The *core_metric* chart demonstrates that the extended context, as well as the second phase of training, has improved the capabilities of the model as a whole. The 2048-context model has a CORE metric of ~0.126, while the 512-context model has ~0.1. This is also evidenced by tests like *winogrande* zero-shot benchmark. While not all individual tests saw improvements, as evidenced by the *winograd* score, the overall metrics suggest that the second phase of training has not only extended the context but has also improved the model's general reasoning and linguistic capabilities as it processes more tokens overall.

3.3.3 Training Stability

The above charts clearly show the success of our training run. The model's performance in the second phase was successful in resuming the training process from step 1000 and reducing the training loss even further to ~3.058.

Part 4: Training Your Final Nanochat

4.1 Final Configuration

- **Selected Architecture:** Nanochat + SwiGLU, num_shards =240, depth=24. Detailed training command: modal run nanochat_modal.py::stage_pretrain --depth=24 --device-batch-size=16 --wandb-run=nano-512-full-p4 --extra="--max-seq-len=512 --num-iterations=5000 --save-every=1000 --model-tag=nano-part4 --warmup-ratio=0.1 --warmdown-ratio=0.5"
- **Justification:** Based on findings in part 2, we confirmed that SwiGLUperforms better than Alibi, and its performance improvements outweigh the decrease. Thus we decided to test the configuration of SwiGLU. For Nanochat we increased the num_shards* and depth accordingly to reflect a more complex model on the full pre-training dataset.

  *while the nanochat has 1822 shards, 240 is enough data for a d24 model at the default

4.2 Scaling Law Prediction

- **Predicted Performance:**

  Chinchilla Law: $L(N, D) \ = e + \frac{a}{N^{\alpha}} + \frac{b}{D^{\beta}}$. (Hoffmann,et al.2022) using the coefficients in the paper:

Coefficients	e	a	b	α	β
Values	1.69	406.4	410.7	0.34	0.28

  Picochat $L(125M, 2B) \ = e + \frac{a}{N^{\alpha}} + \frac{b}{D^{\beta}}$ = 1.69+$\frac{406.4}{125M^{0.34}} + \frac{410.7}{2B^{0.28}} \approx 3.4293$

  Nanochat $L(768M, 12B) \ = e + \frac{a}{N^{\alpha}} + \frac{b}{D^{\beta}}$ = 1.69+$\frac{406.4}{768M^{0.34}} + \frac{410.7}{12B^{0.28}} \approx 2.6957$

  *Frantar's Law (Compression Law) mentioned in Hoffmann,et al.(2022) but not relevant to us in this particular setup.

  Using the published coefficients from Hoffmann et al. (2022) with rounding:

Model	N (params)	D (tokens)	Predicted Loss	Actual Loss	Delta
Picochat + SwiGLU	125M	~2B	3.4293	2.9716	-0.458
Nanochat + SwiGLU	768M	~12B	2.6957	2.5893	-0.106

- **Actual Results(final loss):** picochat+SwiGLU:2.97158; nanochat+SwiGLU: 2.58932
- **Comparison:** While not closely modeling our trends with self derived coefficients, the coefficients in the paper provided a good estimate. Picochat was off by 0.4577 and nanochat off by 0.1064. This shows that our performance followed the chinchilla rule pretty well, this should ensure reasonable learning for the model.

Deriving Our Own Scaling Constants

With two data points (picochat and nanochat), we fit a simple power law of the form L(N) = A · N^(−α) to derive empirical scaling constants from our own training runs:

Training loss scaling:

$$\alpha = ln(\frac{L_{pico}}{L_{nano}}) \div ln(\frac{N_{nano}}{N_{pico}}) = ln(\frac{2.9716}{2.5893}) \div ln(\frac{768M}{125M}) = 0.1379 \div 1.8153 = 0.0760$$

$$A = L_{pico} \times (N_{pico})^{\alpha} = 2.9716 \times (125 \times 10^6)^{0.076} = 12.25$$

$$L(N) \approx 12.25 \cdot N^{-0.076}$$

Validation BPB scaling:

$$\alpha_{bpb} = ln(\frac{0.9064}{0.7777}) \div ln(\frac{768M}{125M}) = 0.1531 \div 1.8153 = 0.0844$$

$$A_{bpb} = 0.9064 \times (125 \times 10^6)^{0.0844} = 4.37$$

$$BPB(N) \approx 4.37 \cdot N^{-0.084}$$

CORE metric scaling (Positive scaling. Larger models achieve higher CORE):

$$\beta_{core} = ln(\frac{0.2186}{0.1366}) \div ln(\frac{768M}{125M}) = 0.47 \div 1.8153 = 0.259$$

$$A_{core} = 0.1366 \times (125 \times 10^6)^{0.259} = 1.09 \times 10^{-3}$$

$$CORE(N) \approx 0.00109 \cdot N^{0.259}$$

Summary of Derived Scaling Laws

Metric	Scaling Law	Pico (125M) Predicted	Pico Actual	Nano (768M) Predicted	Nano Actual Loss
Loss	L(N) = 12.25 · N^(−0.076)	2.972	2.972	2.589	2.589
Val BPB	BPB(N) = 4.37 · N^(−0.084)	0.906	0.906	0.778	0.778

CORE	CORE(N) = 0.00109 · N^(0.259)	0.137	0.137	0.219	0.219

Hypothetical Prediction for Intermediate Scale
Using our derived constants, we can predict the performance of a hypothetical d20 model (~560M params):

Metric	d20 (~560M)	
Prediction Loss	L(560M) = 12.25 × (560M)^(−0.076)	≈ 2.65
Val BPB	BPB(560M) = 4.37 × (560M)^(−0.084)	≈ 0.80
CORE	CORE(560M) = 0.00109 × (560M)^(0.259)	≈ 0.20

Comparison and Discussion
Our estimated exponents (α loss=0.076, α bpb=0.084) are significantly shallower than the Chinchilla exponent (α=0.34). This is to be expected for several reasons:

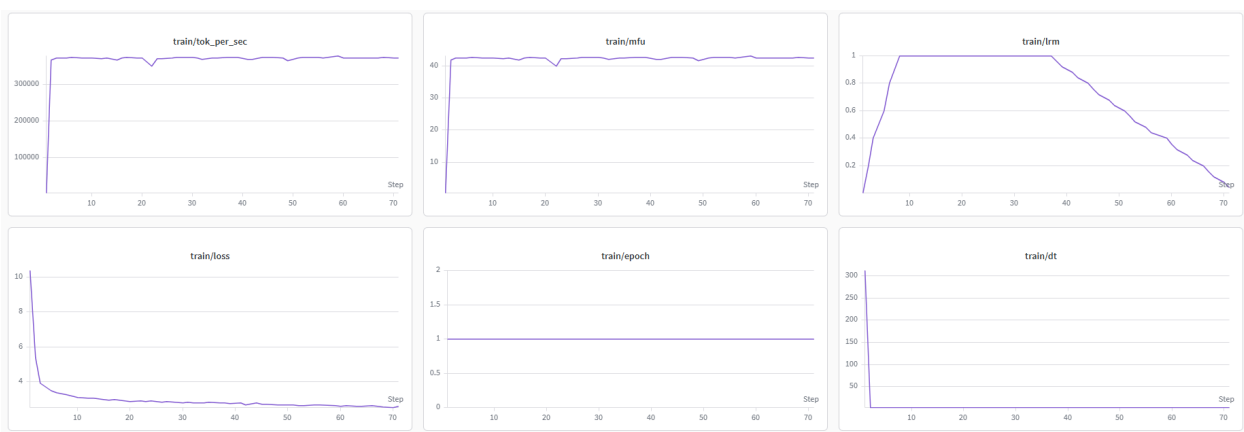
1. Data confound: We have used different sizes of the dataset (40/240 shards) and different iteration counts (2205/5000) for the two models. This causes the simple L(N) fit to average over the model scaling effect and the data scaling effect, leading to a shallower exponent.
2. Architecture modification: SwiGLU changes the capacity per parameter compared to the vanilla Transformer architecture used for Chinchilla.
3. Dataset/tokenizer: The FineWeb-EDU dataset with the custom BPE tokenizer results in different absolute loss values compared to the MassiveText dataset with the SentencePiece tokenizer used for Chinchilla.

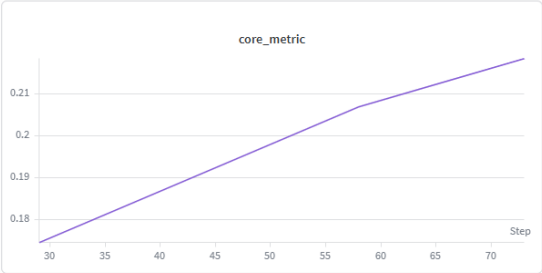
Nonetheless, we find that the prediction of the Chinchilla model and the empirical fit of this work qualitatively agree on the effect of scaling from 125M to 768M parameters, where the loss is reduced by 13-21% while the CORE score is doubled. This verifies the conventional scaling paradigm for the SwiGLU-augmented architecture.

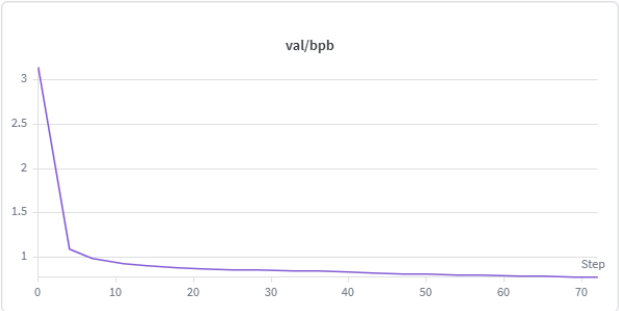
4.3 Final Model Summary

Model	Configuration	Final Loss	Val bpb	CORE metric	MFU	Training Time (min)	Compute Cost ($)
Final Nanochat	Standard nanochat + SwiGLU (Depth 24, 240 Shards, 5000 iterations)	2.58932	0.7777	0.21862	42.5%	4hr 16min	91.23

Commentary: We see the potential demonstrated in the picochat abolition carried over to the full pre-training dataset. Our val bqb and final loss were lowered, in fact 0.77 is quite low for this configuration, showing that our model learned a dense representation of the training dataset, which is reflective of the SwiGLU. We have an increased MFU(42.5%) exceeding the picochat model 1(42.1%), and we also have a nearly doubled CORE (~0.22) compared to the picochat (~0.12). Though our model performance does not match the leaderboard exactly, our results were close. Our val bqb was 0.0294 higher, and our CORE metric was ~0.3 lower with an upward trend. If given slightly more resources, we are likely able to achieve similar performance.







4.4 Emergent Abilities (Nanochat vs. Picochat)

Below are 10 questions/prompts that the fully trained Nanochat can answer coherently, which the smaller Picochat baseline failed or hallucinated on:

Due to repetitions of response, some responses were presented as sets of responses instead of the full log. Sentences in (*italicized*) are comments

1. **“What is 2+2” - simple math problem**
    a. **Nanochat: fails first but contains correct answer**

    {“?”, “2+2 = 2+2”, “2+2 = 4”, “2+2 = 8”}

    b. Picochat:

    {“2+2 is a type of graph that is used to represent the distance between two points, or the distance between two points in a graph.”, “2+2 is a unit of measure for distance, and is used to represent the distance between two points.”, “2+2 is a unit of measure for area, and is used to represent the area of a polygon.”}

2. **“Who is John” - context understanding that john is a human**
    a. Nanochat: Though still hallucination, for instance Illinois Woman's College doesn’t exist, JOHN W. R. GADDLE doesn’t exist etc. However it recognizes the implicit meaning that john is a person.

    “John W. R. Grady

    John W. R. Grady

    JOHN W. R. GADDLE

    (1813-1884) was a member of the first class of the Illinois State NormalSchool, now the University of Illinois at Urbana-Champaign. He was a member of the Illinois General Assembly from 1862-1867.He was the first president of the Illinois Woman's College, a Woman's College, and a president of the Illinois Normal School. He was also an educator, an author, and a lawyer. John W. R. Grady was born in St. Louis, Missouri, on 16 April 1813. He was the second son of John W.

and Mary Grady. He was an educated man, and as a young man he was a teacher in Missouri. He attended the United States Military Academy at West Point, New York, and graduated in 1832. He was brevetted to first lieutenant in the Navy in 1833. He was a member of the 1st Illinois Volunteer Infantry and was elected a captain in the 1st Illinois Volunteers in 1836. He"

b. **Picochat:** "Thesis: 1. The story of the world is a story of the struggle for human existence and the survival of the species. 2. The story of the world is a story of the struggle for human existence and the survival of the species. 3. The story of the world is a story of the struggle for human existence and the survival of the species. 4. The story of the world is a story of the struggle for human existence and the survival of the species. 5. The story of the world is a story of the struggle for human existence and the survival of the species. 6. The story of the world is a story of the struggle for human existence and the survival of the species. 7. The story of the world is a story of the struggle for human existence and the survival of the species. 8. The story of the world is a story of the struggle for human existence and the survival of the species. 9. The story of the world is a story of the struggle for human existence and the survival of the species. 10. The story of the world is a story of the struggle for human existence and the survival of the species. 11. The story of the world is a story of"

3. **"if a=b, b=c, is a=c?" - simple logic**
   a. **Nanochat:** a=b, a=c, b=c, a=b. *(it then goes on to attempt to make an example but fails:)* I have a simple example for you: I have a box of cookies. I put a cookie in the box. I then ask if the box is empty and the person says "yes" or "no". I do the same thing with a box of beans. I ask if the box is empty and the person says "yes" or "no". I do the same thing with a box of eggs. I ask if the box is empty and the person says "yes" or "no". I do the same thing with a box of ice cream. I ask if the box is empty and the person says "yes" or "no". I do the same thing with a box of fish. I ask if the box is empty and the person says "yes" or "no". I do the same thing with a box of chips. I ask if the box is empty and the person says "yes" or "no". I do the same thing with a box of candy. I ask if the box is empty and the person says "yes" or "no". I do the same thing with a box of cookies. I ask if the box is empty and the person says "
   b. **Picochat:** {"The answer is 0.9. How many times have you heard about the "concrete" problem?", "- 0.9: 100 times", "- 0.9: 10 times"}
4. **"can a fish live in the desert" -common sense**
   a. **Nanochat:** coherent and sensical.

      {"a fish living in a desert is a bad idea."}

   b. **Picochat:** "The 20th century saw the demise of the desert tortoise (trevor), with a small population. However, in the 20th century, the desert tortoise was the only species of reptile that could survive in the hot desert climate. The desert tortoise is a species of reptile in the family Amphipoda, which includes the desert tortoise (trevor), the giant tortoises (tortoises), and the desert tortoise (Shelby's tortoise).

The desert tortoise is the largest of the 20 species of reptile. The desert tortoise is also the only member of the family that can live in the hot desert climate.The desert tortoise is a very small species, only living in the desert climate, but it is a very big one. The desert tortoise is about 2 to 4 feet long, and the total length of the tortoise is 3 to 4 feet. The desert tortoise is one of the only species of reptile that can survive in the hot desert climate. The desert tortoise is the only member of the family that can live in the hot desert climate. The desert tortoise is a very small species, only living in the hot desert climate.The desert tortoise is the only member of the family that can live in the hot desert"

5. **"is apple a fruit or vegetable?" (sorry for the grammar mistake)**
    a. **Nanochat: correct classification**

    "The apple is an edible fruit of the flowering plant Malus domestica in the family Rosaceae.It is the most widely cultivated fruit in the world and is thought to have originated in Central Asia. Apples are produced in many parts of the world, including in the United States, Canada, Australia, New Zealand, and South Africa.The apple is a medium-sized fruit, usually red, but occasionally yellow, green, or purple. It is pear-shaped with a green-gold skin and yellow or red flesh. The flesh is sweet and juicy, with a flavor reminiscent of a cross between a lemon and an orange. Apples are widely cultivated and are an important crop in many parts of the world. They are eaten fresh, used in cooking, and used in baking. They are also used in making cider, vinegar, and wines.The best known use of the apple is as a dessert fruit, particularly in European cuisine. In the United States, apples are most often eaten fresh, but they can also be canned or frozen for later use. The apple is also used in cooking, particularly in baking and cider making.The apple is a popular fruit with many people around the world. It is a very nutritious fruit that is high in vitamins and minerals"

    b. **Picochat: no classification**

    "The English word "apple" is derived from the Latin word "apple" which is derived from the Latin word "apple" (apple), meaning "dwarf" or "mature". However, in other languages, "apple" is derived from the Latin word "apple" (apple), which is derived from the Latin word "apple" (apple), meaning "dwarf". Appreciated fruit or vegetable in many cultures is called "apple fruit" (apple fruit), because the fruit is not eaten by the fruit-eating animals, such as the fruit-eating birds.

    - 1 History

    - 2 Appearance

    - 3 Appearance (*goes on counting*)}

6. **"How to convert a decimal number to a percentage?" -instruction giving**

a. **Nanochat: not that much better, still hallucinates but attempts to give a procedure.**

   "How to convert a decimal number to a percentage? Let's consider the question: how to convert a decimal number to a percentage? The answer to this question is simple: the percentage is the sum of all the numbers divided by 100, and the decimal number is the sum of all the numbers divided by 100. (*performance goes down from here*) Here is the formula for converting a decimal number to a percentage: % = (100 / 100) × 100 It is important to note that the formula for converting a decimal number to a percentage is not the same as the formula for converting a decimal number to a fraction. To convert a decimal number to a fraction, you need to multiply the number by 100. This is the same as multiplying a decimal number by 100. To convert a decimal number to a percentage, you need to divide the number by 100. This is the same as dividing a decimal number by 100. As an example, let's consider the decimal number 3.50: To convert this decimal number to a percentage, you would multiply 3.50 by 100: 3.50 × 100 = 31.5% A percentage is a number that represents a fraction. For example"

b. **Picochat: no instruction at all**

   "How to convert decimal numbers to percentage?Decimal numbers are generally known as decimal numbers. This means that they have a decimal place in their place. We can calculate the decimal number with the help of this procedure. How to convert decimal numbers to percentage?Decimal numbers are generally known as decimal numbers. This means that they have a decimal place in their place. This means that they have a decimal place in their place. We can calculate the decimal number with this procedure. In this article, we are going to explore how to convert a decimal number to percentage. We will learn how to convert decimal numbers to percentage.

   Table of Contents

   - 1. How to convert a decimal number to percentage?

   - 2. How to convert a decimal number to a percentage? (keep enumerating)}

7. **"Who was the first president of the United States?" - history**
   a. **Nanochat: correct and comprehensive response**

   "The first president was George Washington. He was the first president of the United States, and he is also the first president of the United States to be born in the United States. He was born on February 22, 1732, in Westmoreland County, Virginia. He was president from 1789 to 1797. He was the first president to be born in the United States and to be the oldest president. He was also the first president to be born in the United States and to be the oldest president. He was born in Westmoreland County, Virginia and was the first president of the United

States. He was born on February 22, 1732, in Westmoreland County, Virginia, and was the first president to be born in the United States. He was the first president to be born in the United States and to be the oldest president. He was born in Westmoreland County, Virginia, and was the first president of the United States. He was born on February 22, 1732, in Westmoreland County, Virginia, and was the first president of the United States. He was born in Westmoreland County, Virginia, and was the first president of the United States. He was born in"

b. **Picochat: incorrect**

"The first president was born in 1793 in the town of Montmartre, Massachusetts, but he was born in the town of New Orleans in 1793. He was the grandson of John Adams, the 17th president of the United States, and his wife, Sarah Frances, a daughter of the 17th president, James Madison. At the age of 18, he was sent to school for a year to attend the first public school in the state. He graduated in 1800, and in 1804, he was elected to the first U.S. Senate. He was elected to the Senate in 1808, and in 1809, he was elected to the first U.S. House of Representatives. In 1811, he was elected to the U.S. Senate, but he was re-elected because he was a Democrat. In 1814, he was elected vice president, and in 1817, he was elected to the U.S. Senate. In 1819, he was elected to the U.S. House of Representatives, but he was elected to the U.S. Senate in 1820, and in 1821, he was elected to the U.S. Senate in 18"

8. **"Which continent is Canada in?" - Geography**
   a. **Nanochat: correct and comprehensive response**

"The answer is: Canada in North America.

Canada is a country in North America, located in the northern part of the continent of North America.

Canada is bordered by the Atlantic Ocean to the west, and the Pacific Ocean to the east. It is the second largest country in the world, after Russia. Canada is a federal parliamentary republic with a monarch as its head of state. The country has a population of 36,795,000 inhabitants, of which 7,354,000 live in the capital city of Toronto (Toronto).The Canadian currency is the Canadian dollar (CAD).Canada is a bilingual country, with French as its official language and English as its second language.The country has a diversified economy, with a large service sector, including the automotive, aerospace, energy, telecommunications, and electronics industries.

Canada is a wealthy country, with a high standard of living.It is a member of the Organization for Economic Cooperation and Development (OECD), the United Nations, and the World Trade Organization (WTO).

Canada is a member of the European Union (EU), the North American Free Trade Agreement (NAFTA), and the North American Free Trade Agreement (NAFTA).Canada is a member of the United Nations, the”

b. **Picochat: incorrect**

“The Canada is the largest island in the world, with an area of 1,336 sq km. Its land area is 1,336 sq km. The land is divided into 4 sub-regions. The sub-regions are:

- The North American continent

- The South American continent

- The North Sea

- The Arctic Ocean

- The Antarctic Ocean

The continent is the largest land area in the world. It's estimated to be 20 million

square kilometres. It's the continent's largest island. The land is divided into 5 continents. The continent is divided into 4 sub-regions:

- North America

- Europe and Asia

- South America

- Antarctica and Antarctica

- The Arctic Ocean

- Antarctica and Greenland

The continent is divided into 4 regions. The North American continent is the largest

by land area. It's estimated to be 1,336 sq km. The South American continent is the largest by land area. It's estimated to be 1,336 sq km. The South American continent is the largest by land area. It's estimated to be 1,336 sq km. The North American continent is the largest by land area. It's estimated to be 1”

9. **"write a sentence about a bird flying" - creativity**
   a. **Nanochat: coherent and has variety**

“1. The birds fly over the garden.2. The birds fly above the house.

3. The birds fly under the trees.4. The birds fly over the sea.

5. The birds fly across the sky.6. The birds fly over the fields.

7. The birds fly under the trees.8. The birds fly over the sky.

9. The birds fly above the clouds.10. The birds fly over the sea.

11. The birds fly over the sun.12. The birds fly over the wind.

13. The birds fly over the moon.14. The birds fly over the rain.

15. The birds fly over the sky.16. The birds fly over the sun.

17. The birds fly over the sea.18. The birds fly over the clouds.

19. The birds fly over the land.20. The birds fly over the sea.

21. The birds fly over the sky.22. The birds fly over the moon.

23. The birds fly over the clouds.24. The birds fly over the sun.

25. The birds fly over the moon.26. The birds fly over the sky.

27. The birds fly over the sea.28. The birds fly over the sun.

29. The”

b. **Picochat: not about flying and only repeats**

“The bird is a bird with a wingspan of 2.5 inches. It is a bird with a wingspan of 2.5 inches.”

10. "write a short story" - testing borders of novelty

a. **Nanochat: fairly coherent until start of story**

“a story and write it in the form of a short story. it is important to know the plot, the characters, the setting and the main events of the story. the story should be written in the past tense. the story should start with an introduction and end with a conclusion. the story should be 100 words long. the story should be written in the form of a short story.

this is the short story that I have chosen to write.

The short story I have chosen is the story of the Three Fishers. The Three Fishers were a group of fishermen who were looking for a fish to catch. They were searching for a fish that was not there. They were looking for a fish that was not there because it was not there. The three fishers were looking for a fish that

was not there because it was not there. They were looking for a fish that was not there because it was not there. There were many reasons why the three fishers were looking for a fish that was not there. The first reason was because the three fishers were looking for a fish that was not there. The second reason was because the three fishers were looking for a fish that was not there because the three fishers were looking for a fish that was not"

b. **Picochat: repetitive in early stage and theme not related to content.**

"Theme: A. G. Ball

Theme: A. G. Ball

In a very short story, the narrator is a man who is a poor man. He is a man who does

not have a wife. When he comes to the house, he is a poor man. He is a poor man because he is a poor man. He is a poor man because he does not have a wife. When he comes

to the house, he is a poor man because he is a poor man. When he comes to the house,

he is a poor man because he is a poor man. When he comes to the house, he is a poor man because he is a poor man. When he comes to the house, he is a poor man because he

is a poor man. When he comes to the house, he is a poor man because he is a poor man

because he is a poor man. When he comes to the house, he is a poor man because he is

a poor man because he is a poor man. When he comes to the house, he is a poor man because he is a poor man because he is a poor man because he is a poor man. When he comes to the house, he"

Overall, Nanochat exhibits simple reasoning, is not too great with numbers, and has some problem of hallucination with words, however we see very good performance with common sense and domain-specific knowledge like history. Compared to picochat however we see drastically increased performance.

Works Cited

Karpathy, A. (2025). *nanochat: The best ChatGPT that $100 can buy* [Computer software]. GitHub. https://github.com/karpathy/nanochat

Leviathan, Y., Kalman, M., & Matias, Y. (2025). *Selective attention improves transformer. In Proceedings of the International Conference on Learning Representations (ICLR 2025)*. https://proceedings.iclr.cc/paper_files/paper/2025/hash/0cf3e7eefb9d643e93e16ff1d94090a7-Abstract-Conference.html

Shazeer, N. (2020). GLU Variants Improve Transformer. arXiv preprint arXiv:2002.05202. https://arxiv.org/abs/2002.05202

Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M. A., Lacroix, T., ... & Lample, G. (2023). Llama: Open and efficient foundation language models. arXiv preprint arXiv:2302.13971. https://arxiv.org/abs/2302.13971

Press, O., Smith, N. A., & Lewis, M. (2021). Train short, test long: Attention with linear biases enables input length extrapolation. arXiv preprint arXiv:2108.12409. https://arxiv.org/abs/2108.12409

Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., ... & Sifre, L. (2022). Training compute-optimal large language models. arXiv preprint arXiv:2203.15556. https://arxiv.org/abs/2203.15556

Warner, B., Chaffin, A., Clavié, B., Weller, O., Hallström, O., Taghadouini, S., Gallagher, A., Biswas, R., Ladhak, F., Aarsen, T., Adams, G. T., Howard, J., & Poli, I. (2025). Smarter, better, faster, longer: A modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference. Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). https://aclanthology.org/2025.acl-long.127/

Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2018). BERT: Pre-training of deep bidirectional transformers for language understanding. arXiv preprint arXiv:1810.04805. https://arxiv.org/abs/1810.04805